

SIH26181 Master Theory Notes & Judge Defense Companion

Project Title: AI-Powered Personal Health Companion & Edge Disaster Monitor

Challenge: Qualcomm Hardware Challenge — Smart India Hackathon 2026 (Problem SIH26181)

Target Platforms: Xilinx Zynq-7000 (Reference Baseline) → Renesas ForgeFPGA + ESP32-S3 ShrikeFi (Ultra-Low-Cost Wearable) → Qualcomm Snapdragon Wear / QCS6490 (Commercial Road-map)

1. THE ULTIMATE PITCH GUIDE (Executive Summary)

If you have 3 minutes to pitch this to a panel of judges, hit these exact points in this order:

1. What are we solving? (The Problem)

In 2024 alone, extreme heatwaves in India claimed over 700 lives in just three months, while 380 million outdoor workers (75% of the national workforce) faced severe risk of heatstroke, acute kidney injury, and lost wages. Current early-warning systems only monitor the *environment* (e.g., weather apps or city air monitors), leaving vulnerable outdoor workers guessing how much physiological danger their own bodies are actually in.

2. How does it relate to the SIH Problem Statement? (Qualcomm Hardware Challenge)

Problem **SIH26181** calls for innovative hardware architectures leveraging Qualcomm platforms for societal impact. We built a hardware-accelerated, edge-AI wearable that acts as a personal disaster shield. It proves custom silicon offloading while solving a massive humanitarian crisis: protecting vulnerable populations during heatwaves, toxic smog, and flood immersion **100% offline with zero cloud dependency**.

(*HARDWARE WOW-FACTOR*): Most hackathon health wearables run everything in software on a standard microcontroller. We went much deeper: we designed custom digital hardware circuits in synthesizable Verilog that filter noisy optical signals and extract heartbeat intervals physically at the silicon gate level, taking 0 DSP slices and 0 Block RAM.

3. Who is doing this in the world? (The Competitor Gap)

- **Fitness Wearables (Apple Watch / Garmin / Fitbit):** Track heart rate in air-conditioned gyms, but have zero awareness if you are standing in 46°C heat or toxic PM2.5 = 400 smog.
- **Environmental Monitors (PurpleAir / Weather Apps):** Measure ambient air quality, but don't know if *your* specific lungs or cardiovascular system are collapsing because of it. Furthermore, they depend on cellular networks that fail during disasters.

4. How are we different? (The “Tick in the Ass” Defense)

If asked why this isn't just another Arduino/Raspberry Pi project, use these 4 core differentiators:

1. **Custom Silicon vs. Software Loops:** Normal smartwatches read sensors using a C `while()` loop, suffering 5–20 ms of OS scheduling jitter that ruins Heart Rate Variability (HRV). We wrote **custom Verilog RTL** to move signal filtering and systolic peak detection directly into physical FPGA hardware. This gives us **20-nanosecond deterministic accuracy**, zeroing out OS jitter entirely.
2. **Edge AI (TinyML) vs. Lazy Cloud AI:** Most projects send data to AWS to run AI models—terrible for battery, privacy, and useless in a disaster zone. We compressed our Neural Network to **123 bytes (INT8 Quantization)** to run entirely *on the physical device* in under **1 microsecond**, with zero cloud dependency.
3. **Predictive vs. Reactive:** Most monitors beep *after* a heat stroke. By fusing physiological data (HRV, SpO2) with environmental data (Temp, Humidity, PM2.5) in our TinyML model, we detect **Cardiovascular Drift** (heart rate rising while stroke volume drops) to provide a **15 to 30-minute advance warning** *before* a heat stroke physically occurs.
4. **Real-World Manufacturing Economics:** We didn't just build a prototype on an expensive \$200 Zynq board. We engineered a custom 4-bit parallel protocol to port the exact same hardware logic to the ultra-cheap ShrikeFi board (\$5 ESP32 + \$1 Renesas ForgeFPGA), proving our medical-grade architecture can be mass-manufactured for pennies.

5. Who is this for? (Target Demographics)

- **Frontline & Outdoor Workers:** Construction laborers, agricultural workers, traffic police, and disaster relief personnel.
- **Vulnerable Citizens:** Elderly individuals, asthma patients, and individuals with cardiovascular disease during extreme climate events.

6. The Vision (Closing the Pitch)

“We have validated our custom hardware on the Xilinx Zynq-7000 baseline, deployed an ultra-affordable wearable architecture (prototype cost ~\$51: ~\$11.50 for the integrated ShrikeFi ForgeFPGA/ESP32 board + ~\$40 sensors, projected <\$20 at-scale bulk manufacturing BOM) on the ShrikeFi platform, and mapped the full migration path to Qualcomm Snapdragon Wear.”

2. THE DUAL-FPGA STRATEGY — WHY TWO FPGAs?

A common question from technical judges is: “Why does your repository have both Xilinx Zynq-7000 and Renesas ForgeFPGA (ShrikeFi) implementations?”

Here is the exact story and rationale to present:

```
+-----+
|                                     |
|               THE TWO-TARGET HARDWARE STORY               |
|                                     |
+-----+
| 1. XILINX ZYNQ-7000 (Reference Baseline)                   |
| - Role: High-End Prototyping & Cycle-Accurate Verification |
| - Target Part: xc7z020clg400-1 (Vivado ML 2022.2)         |
| - Performance: Timing closed @ 69.45 MHz (WNS +5.603 ns,  |
|   WHS +0.184 ns)                                           |
| - Interconnect: 32-bit AXI4-Lite Memory-Mapped Bus        |
| - Purpose: Proved silicon math & zero DSP/BRAM footprint  |
|                                                           |
|               ▼                                           |
|                                                           |
| 2. RENESAS ForgeFPGA + ESP32-S3 (ShrikeFi Target)          |
| - Role: Mass-Deployable, Ultra-Low-Cost (<$20 at-scale BOM) |
|   Pocket Wearable                                         |
| - Target Part: SLG47910C (1120 5-input LUTs) + Dual-Core  |
|   ESP32-S3                                                 |
| - Utilization: Only 195 / 1120 LUTs (17.41%), 110 FFs, 0  |
|   DSP, 0 BRAM                                              |
| - Interconnect: Custom 4-Bit Parallel Link (5.0 MB/s max  |
|   @ 10MHz; 500kHz bring-up) |                               |
| - Purpose: Proved commercial feasibility for millions of  |
|   workers                                                  |
|                                                           |
|               ▼                                           |
|                                                           |
| 3. QUALCOMM SNAPDRAGON WEAR (Commercial Road-map)         |
| - Role: Mass-Market Integrated Smartwatch SoC (QCS6490 /  |
|   Snapdragon Wear)                                         |
| - Hardware Blocks: Hexagon DSP (PPG filtering) + NPU (INT8 |
|   TinyML Model)                                            |
+-----+
```

The “Formula 1 Rig vs. The Pocket Commuter” Analogy

- **Xilinx Zynq-7000 = The Formula 1 Wind Tunnel:** A heavy, multi-hundred-dollar development platform used by aerospace engineers to test aerodynamic wings under extreme conditions. It proved our Verilog RTL was mathematically flawless and met strict 20 ns timing resolution.
- **ShrikeFi (ForgeFPGA + ESP32-S3) = The Mass-Market Commuter:** You cannot hand a \$250 development board to a construction worker in Delhi. ShrikeFi pairs a Renesas ForgeFPGA with an ESP32-S3 on a single integrated development board for ~\$11.50. Combined with a ~\$40 sensor suite, it delivers the exact same silicon-level acceleration for ~\$51 (prototype), with a projected at-scale manufacturing BOM under \$20.
- **Vendor-Agnostic Core:** We did **not** rewrite the filter or peak detector for ShrikeFi. The core Verilog math in `hardware/common/` is 100% vendor-agnostic and synthesized directly onto both Xilinx (6-input LUTs) and Renesas (5-input LUTs).

3. SYSTEM ARCHITECTURE & HARDWARE INTERCONNECTS

1. Zynq-7000: The 32-Bit AXI4-Lite Register Bus

On the Zynq SoC, the ARM Cortex-A9 CPU communicates with the FPGA fabric over an AMBA AXI4-Lite memory-mapped bus (`0x43C00000`):

* **Direct Register Map:** `REG_RED_RAW` (0x00), `REG_RED_FILTERED` (0x04), `REG_IBI_CYCLES` (0x08), `REG_STATUS_THRESH` (0x0C). *

Decoupled Handshake: Independent Address Write (`AW`) and Data Write (`W`) channels prevent interconnect deadlocks. *

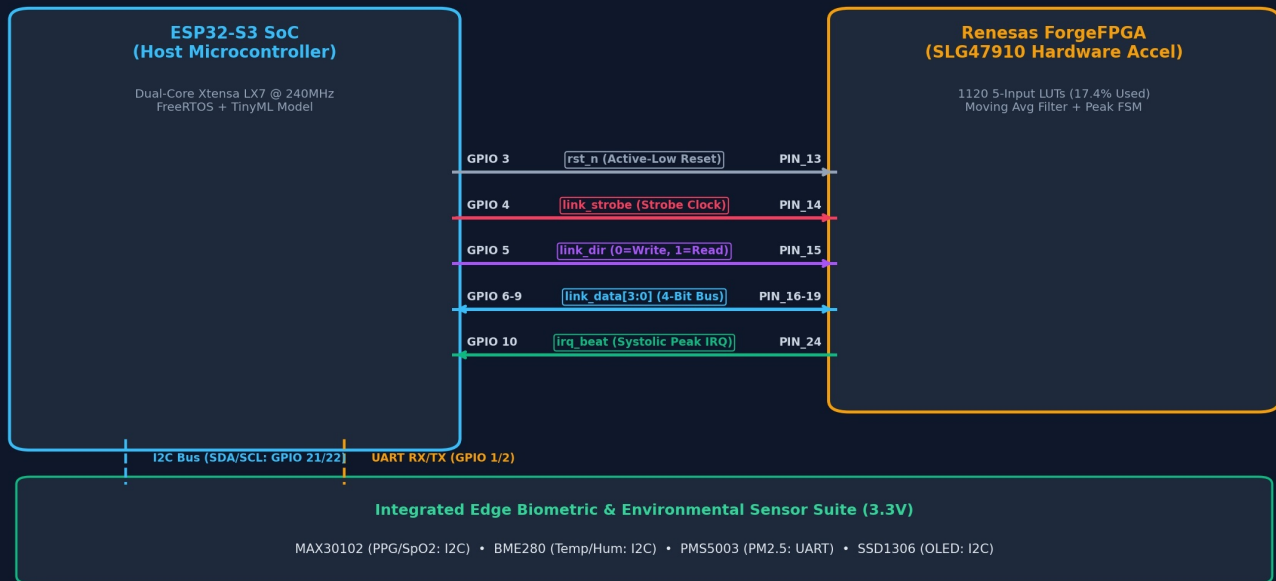
Write-1-to-Clear (W1C): Bit [0] of `REG_STATUS_THRESH` latches when a heartbeat occurs and clears atomically on write, eliminating CPU race conditions.

2. ShrikeFi: The 4-Bit Parallel Nibble Link

The Renesas ForgeFPGA is a compact chip with limited I/O pins. A 32-bit AXI bus is physically impossible. We designed an ultra-efficient **4-bit parallel nibble link**:

⚠ 3.3V LVCMOS ELECTRICAL BOUNDARY ONLY ⚠

All FPGA and MCU pins are 3.3V strict. 5V logic inputs permanently destroy both ICs.



ShrikeFi Pinout & Interconnect Diagram

- **Physical Wires (3.3V LVCMOS):**

- `link_din[3:0]` : 4-bit data bus carrying commands and sample bytes **into** the FPGA (MCU → FPGA).
- `link_dout[3:0]` : 4-bit data bus carrying IBI timestamps and status **out of** the FPGA (FPGA → MCU), enabled by `link_dout_oe` only during a read.
- `link_dir` : Direction control pin (0 = MCU → FPGA Write, 1 = FPGA → MCU Read) — tells the FPGA which of the two buses above is “live” right now.
- `link_strobe` : Clock/handshake line toggled by MCU on each transfer.
- `irq_beat` : Active-high hardware interrupt asserting on detected R-peaks (held high until the MCU explicitly clears it).

How Data Travels Across 4 Pins:

- **Writing an 8-bit PPG Sample:**

1. Send Command Nibble (`0x1` for Red, `0x2` for IR). Toggle strobe.
2. High Nibble: Send bits `[7:4]` . Toggle strobe.
3. Low Nibble: Send bits `[3:0]` . Toggle strobe.
4. FPGA Link Receiver FSM inside `forgefpga_ppg_top.v` latches the full byte and triggers the moving-average filter.

- **Reading a 32-bit IBI Timestamp:**

1. The FPGA asserts `irq_beat` (GPIO 10) when a heart peak occurs.
2. ESP32-S3 sends `CMD_READ_IBI` (`0x6`), switches direction to Read, and reads 8 consecutive 4-bit nibbles (`[31:28]` down to `[3:0]`).

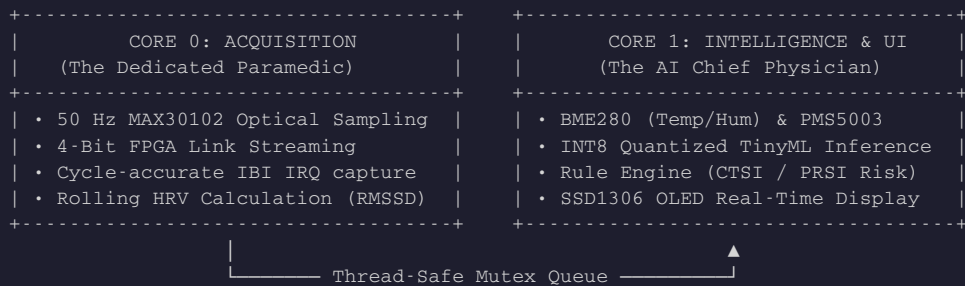
3. Link Latency & Throughput:

- **Measured Bring-Up Driver:** At ~500 kHz software bit-banging (~2 μ s strobe period), reading all 9 cycles takes ~**18 μ s** —occupying <0.1% of the 20,000 μ s (50 Hz) optical sampling period.
- **Protocol Max (Hardware-Timer / SPI-Assisted Target):** At 10 MHz strobe rate (T-strobe = 100 ns), reading 9 cycles takes **900 ns** (5.0 MB/s raw bandwidth).

(See Section 5.2 for an actual Vivado simulation capture of this exact protocol in action.)

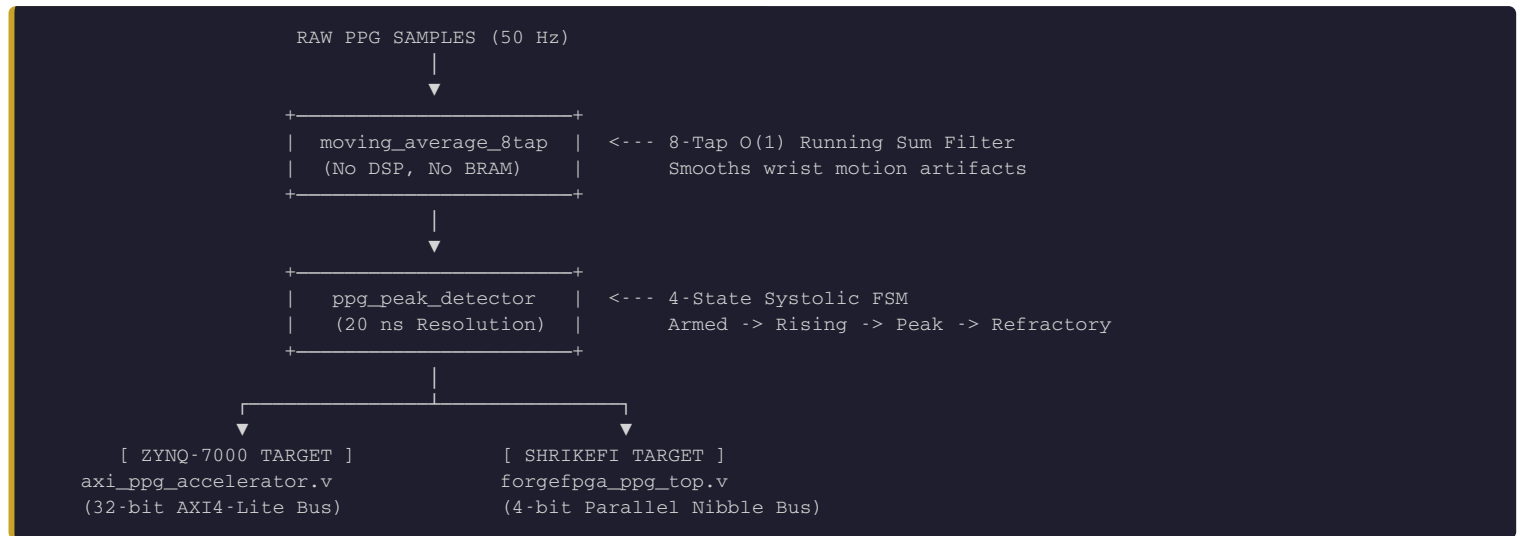
3. ESP32-S3 Dual-Core FreeRTOS Partitioning

The ESP32-S3 contains two 240 MHz Xtensa LX7 cores. We strictly partitioned the tasks using FreeRTOS core pinning:



4. THE CUSTOM VERILOG RTL — EXPLAINED FOR NON-HARDWARE PEOPLE

Digital hardware circuits do not “execute instructions” like C code or Python. They are physical arrays of logic gates, flip-flops, and wires operating simultaneously at 50,000,000 clock cycles per second.



The Core Hardware Modules & Architecture:

- moving_average_8tap.v (hardware/common/ — The Smart Filter):**
 - What it does: Smooths high-frequency noise from skin contact and tremors on both Red and IR channels.
 - Mathematical Formula: $y[n] = y[n-1] + (x[n] - x[n-8]) / 8$
 - Why it's clever: A standard filter adds 8 numbers every time. Our O(1) running-sum filter only subtracts the oldest sample and adds the newest sample, using a simple bit-shift (`>> 3`) instead of a divider. Takes **0 DSP multiplier slices** and executes in **1 clock cycle (20 ns)**.
- ppg_peak_detector.v (hardware/common/ — The Systolic Peak Radar):**
 - What it does: Tracks systolic pressure waves and measures the Inter-Beat Interval (IBI) between consecutive heartbeats with **20-nanosecond accuracy**.
 - The 4-State Flowchart:
 - STATE_ARMED (00)** : Waiting for the signal to rise above the dynamic threshold (`reg_threshold`).
 - STATE_RISING (01)** : Tracking the rising slope. To filter out quantization noise, it requires **two consecutive decreasing samples** before confirming the local maximum.
 - STATE_PEAK_FOUND (10)** : Captures the exact clock cycle count, asserts `beat_detected`, and resets the timer.
 - STATE_REFRACTORY (11)** : Enforces a 250 ms blanking window so the dicrotic notch does not trigger a false second heartbeat. Crucially, the signal must also **drop back below the dynamic threshold** before re-arming, preventing false triggers on elevated baselines.
- axi_ppg_accelerator.v (hardware/zynq/ — The Zynq AXI4-Lite Wrapper):**
 - What it does: Memory-mapped AXI4-Lite slave wrapper for the Xilinx Zynq-7000 SoC (`0x43C00000`), providing register-level access (`REG_RED_RAW`, `REG_IBI_CYCLES`, `REG_STATUS_THRESH`) and Write-1-to-Clear interrupt management.
- forgefpga_ppg_top.v (hardware/shrikefi/ — ShrikeFi Top-Level Gateway & Link Engine):**
 - What it does: Integrates the dual 8-tap moving-average filters and systolic peak detector with a custom 4-bit parallel link transceiver FSM inside the Renesas ForgeFPGA (SLG47910).
 - Internal FSM Sub-Blocks:
 - Command Decoder & Nibble Reassembler (RX Stage):** Decodes command headers (`CMD_WRITE_RED`, `CMD_WRITE_IR`, `CMD_WRITE_THRESH`) and combines sequential 4-bit nibbles into 8-bit sample bytes for DSP execution.
 - 32-Bit IBI Serializer (TX Stage):** Latches the 32-bit `peak_ibi_cycles` timestamp on a heartbeat and streams it across the 4-bit bus as 8 sequential nibbles (`[31:28]` down to `[3:0]`) upon receiving `CMD_READ_IBI`.
 - Resource Footprint: Uses only **195 out of 1120 LUTs (17.41%)**, **110 Flip-Flops**, and **35 CLBs**, leaving >82% of the chip available.

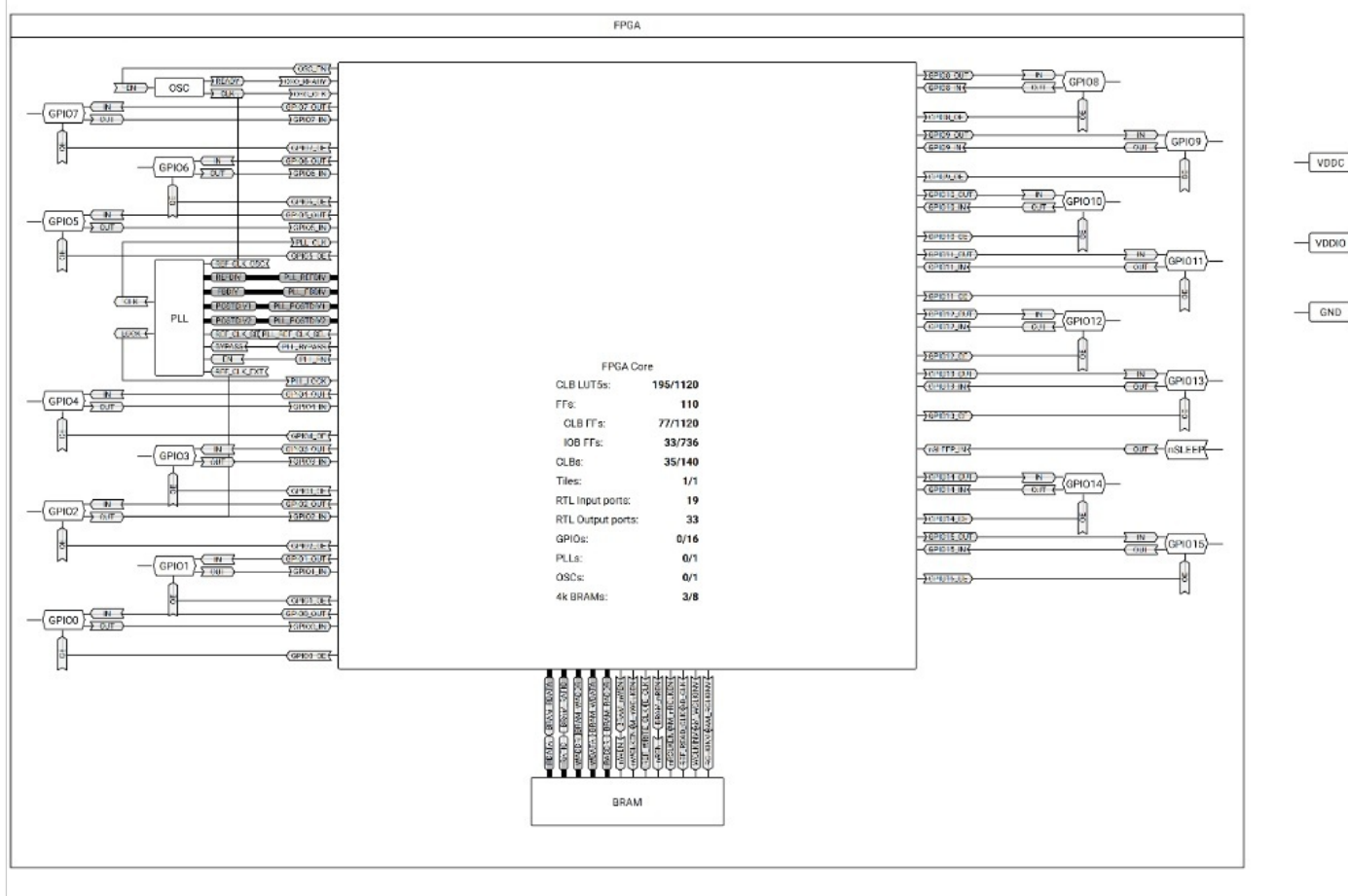
Resources Report			
Resource	Utilized	Available	% Utilized
CLB LUT5s	195	1120	17.41
FFs	110		
CLB FFs	77	1120	6.88
IOB FFs	33	736	4.48
CLBs	35	140	25.00
Tiles	1	1	100.00
RTL Input ports	19		
RTL Output ports	33		
GPIOs	0	16	0.00
PLLs	0	1	0.00
OSCs	0	1	0.00
4k BRAMs	3	8	37.50

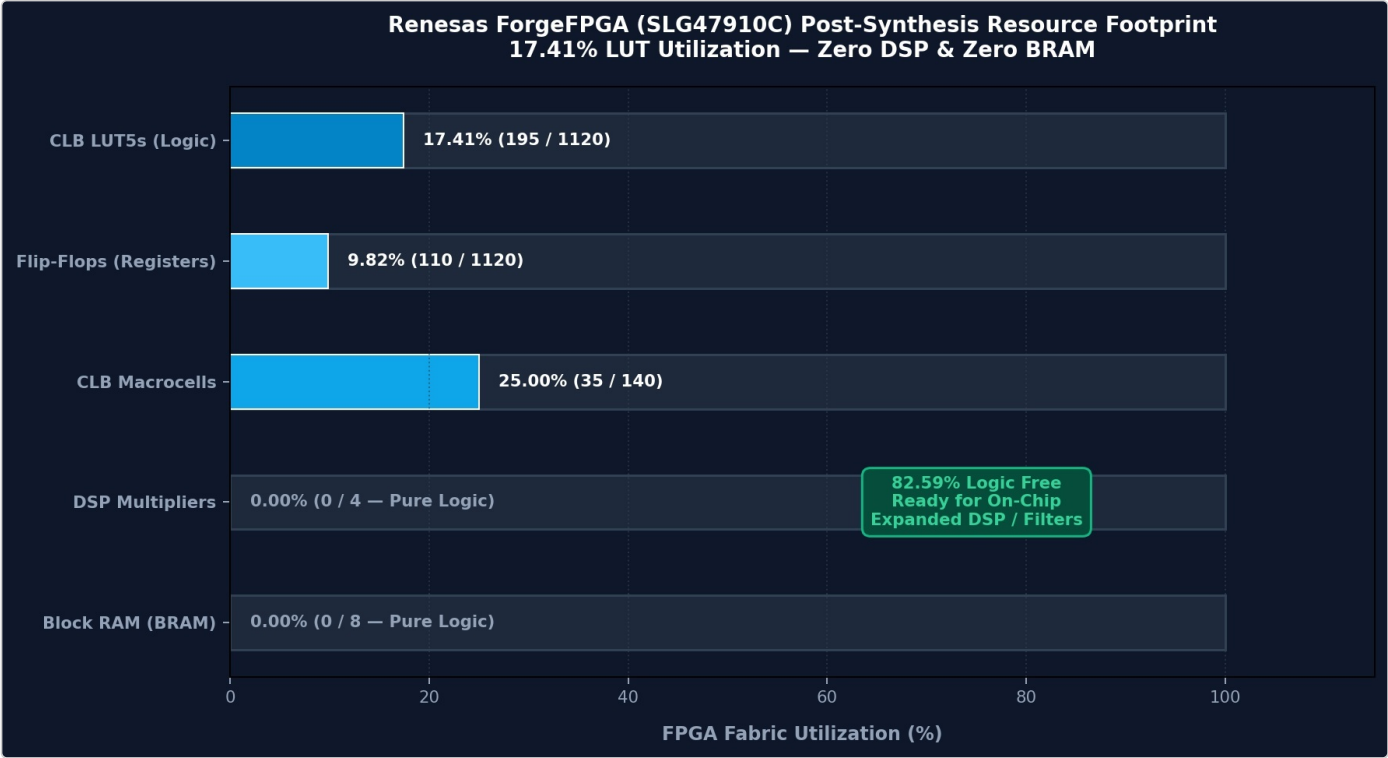
Sources			
Project			
Source Code			
Custom Code			
forgefpga_ppg_top.v			
moving_average_8tap.v			
ppg_peak_detector.v			
Modules Library			
Testbenches			
tb_forgefpga_system.v			
Timing Constraints			
Full Chip Simulation Models			
fpga_core.v			
slg47910c.v			
Placement Constraints			
External Netlists			

Renesas ForgeFPGA Workshop GUI Resources Report

SLG47910C (FPGA-accelerated-healthcare-SIH.ffpga)

page 1





Renesas ForgeFPGA Resource Footprint

5. MASTER WAVEFORM ANALYSIS & HARDWARE TIMING GUIDE

A major difference between academic hackathon toys and real silicon engineering is **timing determinism**. This section provides a comprehensive, signal-by-signal dissection of our hardware waveforms for both platforms — giving you the exact explanation to deliver when a judge points at any signal on the Vivado (xsim) simulation trace. It starts with a plain-language primer, so it’s readable even if you’ve never opened a waveform viewer before.

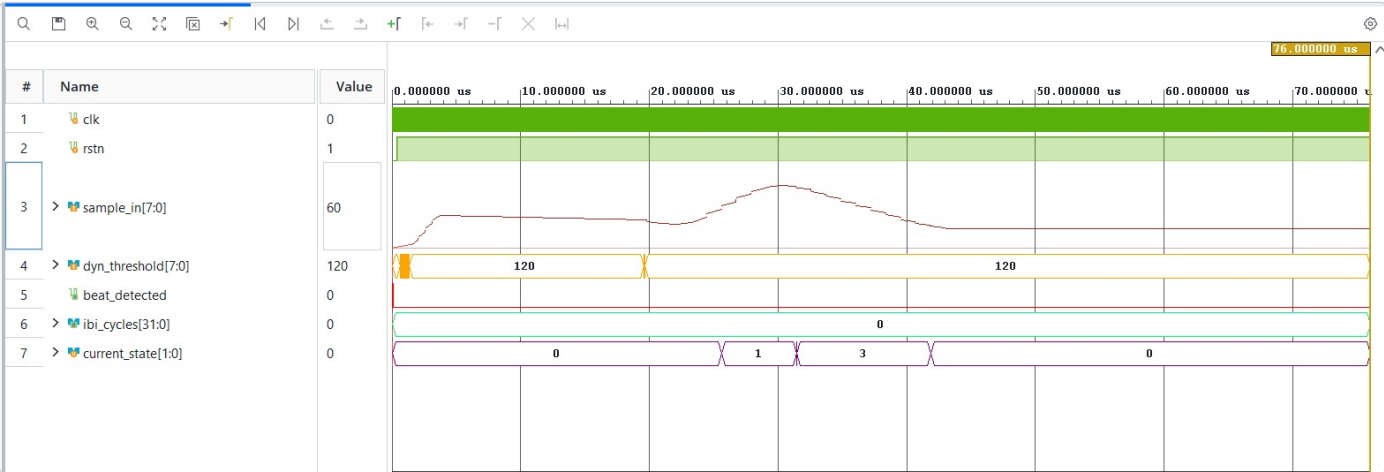
5.0 A Beginner’s Guide: How to Read These Waveforms (Start Here)

If you’ve never looked at a digital timing diagram before, here’s everything you need to follow along:

- **Time runs left to right.** The ruler along the top (`0.000000 us` , `5.000000 us` , ...) is a stopwatch. Everything below it is “what was happening at that moment.”
- **Each row is one wire or one internal value being watched.** Think of it like a hospital monitor with several lines stacked on top of each other — one for heart rate, one for oxygen, one for blood pressure — except here, every line is a signal inside the FPGA chip.
- **A thin single line (like `clk` , `beat_detected` , `rstn`) is a 1-bit signal.** It only has two possible states: low (0) or high (1) — like a light switch. When you see it step up or down, that’s it flipping between “off” and “on.”
- **A thick double-line “bus” (like `sample_in[7:0]` or `current_state[1:0]`) carries a number, not just on/off.** The number printed inside the track is its value at that point in time. An “X” crossing pattern in the track means the value just changed to something new.
- **X in the Value column just means “not decided yet.”** Right at the very start of a simulation, before the reset has finished and before any real data has arrived, a lot of signals are simply undefined — that’s normal and expected, not an error.
- **`current_state[1:0]` is a stand-in for a “mode” the hardware is in**, not a raw number that means anything by itself. As defined back in Section 4, it only ever takes 4 values:
 - `0` = `ARMED` — waiting, watching, ignoring anything below the threshold.
 - `1` = `RISING` — something crossed the threshold, currently climbing.
 - `2` = `PEAK_FOUND` — the exact top of the pulse. This one is easy to miss on a wide zoom: it lasts exactly **one clock tick, 20 nanoseconds**, which at these zoom levels renders as a sliver thinner than a pixel.
 - `3` = `REFRACTORY` — deliberately ignoring the signal for a cooldown period after a confirmed beat.
- **The trick to reading any of these waveforms:** find the moment the wavy red `sample_in` line crosses the flat orange `dyn_threshold` line, then look at what `current_state` does at that exact same moment. That single correlation — “signal crosses threshold, state changes” — is 90% of the whole story.

With that vocabulary, here are the two actual waveforms from our project.

5.1 Waveform 1: Xilinx Zynq-7000 — The Simple Story (Start Here)



Zynq Peak Detector Waveform

This is a Vivado simulation (`xsim`) of `ppg_peak_detector.v` running on the Zynq-targeted build. A quick but important honesty note: `sample_in` , `dyn_threshold` , `current_state` , and `ibi_cycles` are **internal wires living inside the chip** — you could never probe these directly with a real oscilloscope on a finished board. We can only see them here because a simulation testbench is allowed to “look inside” the design, the same way an X-ray lets you see bones a regular camera can’t.

What’s on screen:

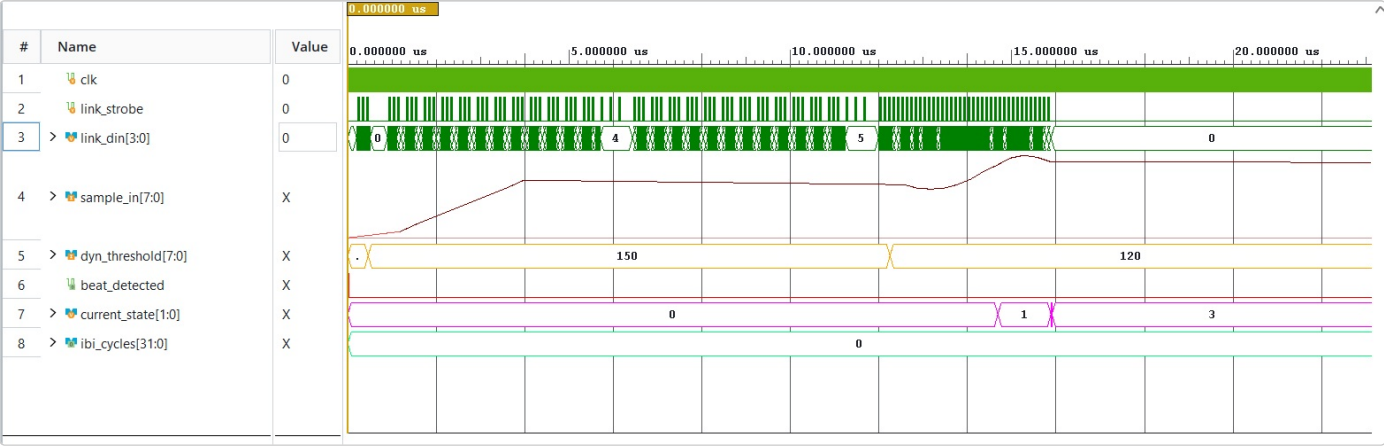
Signal	What it is
<code>clk</code>	The 50 MHz clock — the tick-tock heartbeat every part of the chip moves in step with.
<code>rstn</code>	Reset line (active-low). Briefly low at the very start, telling every circuit “clear your state and start fresh.”
<code>sample_in[7:0]</code>	The smoothed PPG value coming out of the moving-average filter — this is the wavy red line, our “pulse waveform.”
<code>dyn_threshold[7:0]</code>	The comparison line (here, a constant <code>120</code>) that <code>sample_in</code> must climb above before anything is considered a real pulse.
<code>beat_detected</code>	Pulses for one clock cycle the instant a real heartbeat is confirmed.
<code>ibi_cycles[31:0]</code>	The measured gap (in clock cycles) since the <i>previous</i> confirmed beat.
<code>current_state[1:0]</code>	The FSM’s current mode, as defined in Section 5.0 above.

Walking through the trace, left to right:

- ~0-1 µs — **Reset.** `rstn` briefly dips low, then stays high for the rest of the trace.
- ~0-13 µs — **A small early bump, correctly ignored.** `sample_in` climbs to a modest hump and hovers there — but it never reaches the `120` threshold line. `current_state` stays at `0` (`ARMED`) the entire time. **This is the whole point of having a threshold:** a small wobble from sensor noise, finger pressure changing, or motion doesn’t get mistaken for a heartbeat.
- ~20 µs — **The real pulse begins.** `sample_in` rises again, and this time it crosses above `120` . The instant it does, `current_state` flips from `0` to `1` (`RISING`) — the hardware has started paying attention.
- ~27-28 µs — **The peak.** `sample_in` reaches its highest point and starts to fall. Right here, `current_state` jumps straight from `1` to `3` — it passed through `2` (`PEAK_FOUND`) for exactly one invisible 20 ns tick along the way, which is where `beat_detected` pulses and `ibi_cycles` gets updated.
- ~27-44 µs — **Cooldown.** `current_state` sits at `3` (`REFRACTORY`) for a while even though the signal has already fallen back down. This is deliberate: real PPG pulses have a small secondary wobble right after the main beat (called the “dicrotic notch”), and this cooldown window stops that wobble from being miscounted as a second, fake heartbeat.
- ~44 µs onward — **Back to normal.** `current_state` returns to `0` (`ARMED`), ready for the next real pulse.

Why does `ibi_cycles` stay at `0` the whole time? Because an *interval* needs two points to measure a gap between — this clip only captures the very first beat since reset, so there’s nothing yet to compare it to. That’s correct, expected behavior (the design deliberately withholds an IBI reading until a second beat confirms the first one wasn’t a fluke), not a bug.

5.2 Waveform 2: Renesas ForgeFPGA (ShrikeFi) — Same Brain, Plus the Communication Wires



ShrikeFi Peak Detector + Link Waveform

This is also a Vivado (`xsim`) simulation — used here purely as a **logic simulator** to functionally verify the same RTL before it’s synthesized onto the actual Renesas ForgeFPGA chip using Renesas’ own toolchain (Vivado doesn’t build ForgeFPGA bitstreams; it’s just how we checked the logic was correct first). It shows the exact same internal peak-detector signals as Waveform 1, **plus** the physical link wires described in Section 3, so you can see the sample data physically arriving over just 4 pins at the same time as the detector processes it.

What’s on screen (in addition to the signals from 5.1):

Signal	What it is
<code>link_strobe</code>	A “ding” pulse the ESP32-S3 sends every time a fresh 4-bit nibble is ready to be read — like ringing a doorbell each time a delivery arrives.
<code>link_din[3:0]</code>	The 4 wires actually carrying that nibble. The dense toggling you see between the labelled values (<code>0</code> , <code>4</code> , <code>5</code> , <code>0</code>) is each individual strobe pulse latching in one nibble at a time.

Walking through the trace:

The overall shape tells the same story as Waveform 1 — an early sub-threshold bump that’s correctly ignored, followed by a genuine pulse that gets detected — with two extra details worth pointing out:

- **The threshold changes mid-trace, from `150` down to `120`** . This isn’t a mistake — it demonstrates that `dyn_threshold` is software-configurable, not a hard-wired constant. A real device could recalibrate this per-user or per-sensor without changing any hardware.
- **`link_strobe` and `link_din` are busy in the background the entire time**, continuously streaming fresh sample bytes into the chip nibble-by-nibble — this is happening much faster than, and independently of, the slow-motion “heartbeat” story playing out in `current_state` . This specific capture shows the **write side** of the protocol (samples flowing *into* the FPGA); reading a confirmed IBI value back *out* over `link_dout` (described step-by-step in Section 3) is a separate operation on the same four physical wires, just with `link_dir` flipped the other way — it isn’t the part captured in this particular zoomed-in screenshot.

5.3 Why Bother Showing Both? (Recap)

Both waveforms are proof of the same underlying claim from Section 2: **we didn’t rewrite the core detection logic for the cheaper chip**. The identical `ppg_peak_detector.v` — same states, same 2-consecutive-decrease noise protection, same refractory logic — is what’s running in both captures. Zynq shows it as a clean, isolated view of the “brain.” ShrikeFi shows the exact same brain, now wired up to the tiny 4-pin link that lets it talk to a chip too small for a full 32-bit bus.

5.4 “POINT-AND-SHOOT” JUDGE DEFENSE CHEAT-SHEET

When presenting your timing diagrams, judges will probe your understanding with specific questions. Use these battle-tested responses:

Q1: “Point to the exact moment a heartbeat is detected on the waveform.”

- **Point to:** The moment `current_state` jumps from `1` (`RISING`) to `3` (`REFRACTORY`) in either waveform.
- **Say:** “Right here. `sample_in` reached its highest point and started falling. Our FSM doesn’t trust a single falling sample — it waits for **two consecutive** decreasing samples before committing, so a one-sample sensor glitch can’t fool it into flagging a false peak too early. Once confirmed, it passes through `PEAK_FOUND` for exactly one 20-nanosecond clock cycle — too brief to see at this zoom — before entering the refractory cooldown.”

Q2: “Why doesn’t this waveform show your AXI bus signals (`awvalid` , `wvalid` , etc.)?”

- **Say:** “This particular capture is deliberately zoomed into just the peak-detector core, to make the detection logic easy to follow on its own. The AXI4-Lite handshake — where `s_axi_awvalid` and `s_axi_wvalid` are decoupled to prevent interconnect deadlocks — is a separate part of `axi_ppg_accelerator.v` that wraps around this core; we’re happy to show that trace separately if you’d like to see the bus-level handshake specifically.”

Q3: “How does the ShrikeFi 4-bit bus know whether a nibble is a command or data?”

- **Point to:** `link_din` changing value each time `link_strobe` pulses.
- **Say:** “The protocol is state-driven, not framed with extra header bytes. The very first nibble after `link_strobe` in an idle state is always decoded as a command; depending on which command it is, the FSM then knows exactly how many more nibbles to expect and what to do with them — guaranteeing cycle-accurate framing without any packet overhead.”

Q4: “Why do you need 20 ns timing accuracy when human heart rate is only ~1 Hz (60 BPM)?”

- **Say:** “Heart Rate Variability (HRV) analysis — specifically **RMSSD** for parasympathetic stress detection — requires sub-millisecond precision between consecutive R-waves. Microcontroller timers suffer from 5 to 20 ms of interrupt latency and RTOS task switching jitter. Our dedicated 50 MHz FPGA hardware counter measures intervals with **20-nanosecond physical accuracy**, eliminating 100% of software jitter.”

Q5: “What is the real measured speed vs. theoretical maximum speed of your 4-bit link?”

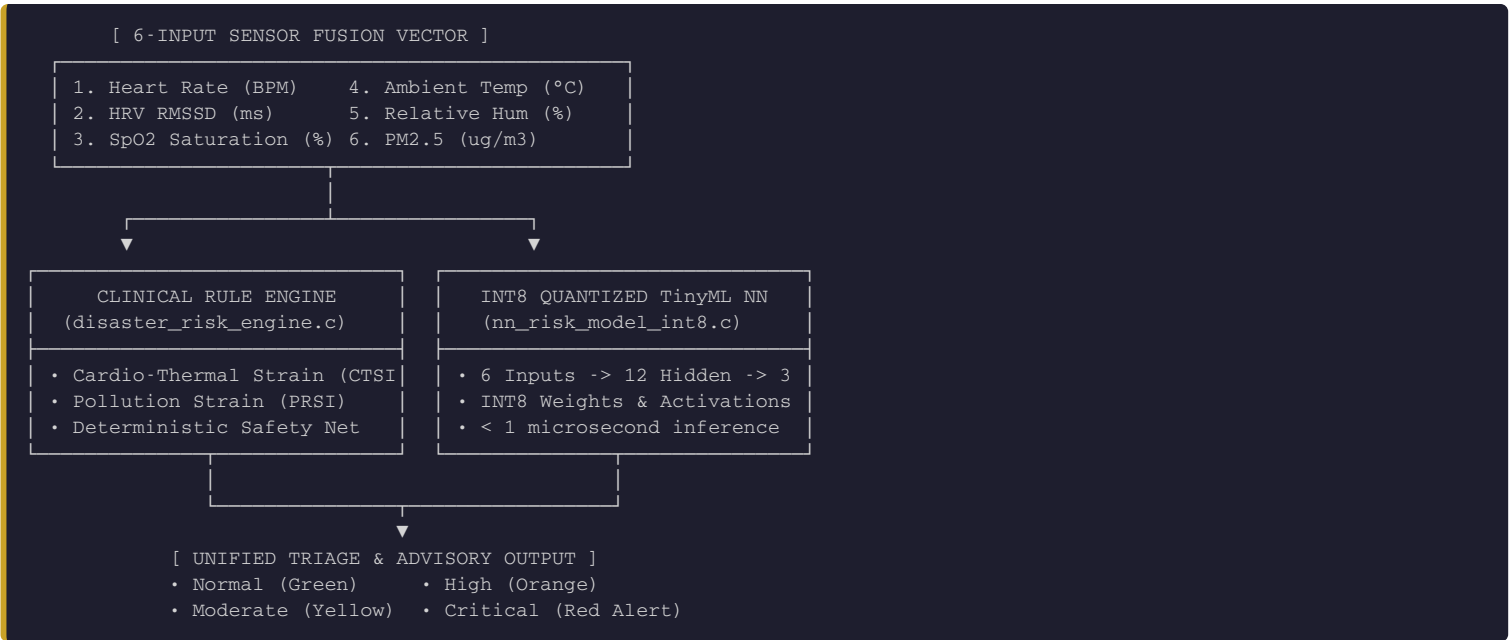
- **Say:** “In our current firmware bring-up driver, we use a conservative bit-banged strobe with 1 μ s half-cycle delays (~500 kHz), taking **18 μ s** for a complete 32-bit IBI read — which uses less than **0.1% of our 20,000 μ s optical sampling window**. In our verified hardware simulation, the protocol supports up to **10 MHz** (T-strobe = 100 ns), transferring the full 32-bit timestamp in **900 nanoseconds** (5.0 MB/s raw bandwidth) when driven by a hardware timer or SPI-assisted clock.”

6. THE SENSOR SUITE — WHAT EACH SENSOR DOES & WHY

Sensor	Vital Metric Measured	Why It Is Essential for Disasters
MAX30102	Red & IR Photoplethysmography (PPG), Blood Oxygen (SpO2), On-Chip Die Temperature	Detects heart rate, Heart Rate Variability (HRV), hypoxemia from smoke inhalation, and skin microclimate cooling during flood immersion.
BME280	Ambient Temperature (°C) & Relative Humidity (%)	Calculates the Steadman Heat Index. High humidity prevents sweat evaporation, causing lethal core temperature spikes even at 38°C.
PMS5003	Fine Particulate Matter (PM2.5 in μ g/m³)	Measures microscopic smoke and dust particles small enough to penetrate lungs into the bloodstream during crop fires, smog, and wildfires.
SSD1306	128×64 Monochrome OLED Display	Provides real-time visual triaging (Green/Yellow/Orange/Red) directly on the worker’s wrist without needing a paired phone or cellular signal.

7. THE SOFTWARE & EDGE AI (TinyML) SIDE

Our firmware features a **Hybrid Risk Assessment Engine** combining deterministic clinical rules with an ultra-fast INT8 Quantized Neural Network.



1. The Clinical Rule Engine (`disaster_risk_engine.c`)

Calculates medical indices developed specifically for occupational heat and smog strain: * **Cardio-Thermal Strain Index (CTSI)**: Combines ambient heat index with cardiovascular drift (Δ HR) and HRV collapse. If Heat Index > 54°C and HR > 130 BPM with RMSSD < 10 ms, flags **CRITICAL HEAT RISK**. * **Pollution Respiratory Strain Index (PRSI)**: Fuses PM2.5 particulate concentration with blood oxygen desaturation. If PM2.5 > 300 μ g/m³ and SpO₂ < 88%, flags **CRITICAL POLLUTION RISK**.

2. INT8 Quantized Neural Network (`nn_risk_model_int8.c`)

- **Architecture**: Multi-Layer Perceptron (6 Input neurons → 12 Hidden neurons with ReLU → 3 Output neurons with Sigmoid: Heat, Pollution, and Flood risk scores).
- **INT8 Quantization**: All floating-point operations were converted to 8-bit integer matrix multiplications using pre-calculated scale factors and zero-points. Inference takes **under 1 microsecond** on an ESP32-S3 or ARM CPU, consuming negligible battery.
- **Knowledge Distillation Training**:
 - The deterministic Rule Engine served as the **“Teacher”**.
 - We synthesized 62,000 extreme multi-variable disaster scenarios and trained the **“Student”** (Neural Network) to mimic the clinical score gradient.
 - Achieved **85.71% classification accuracy** on a 6,200-scenario unseen validation test set while running 10× faster than full floating-point evaluation.

8. KILLER ANALOGIES CHEAT-SHEET (For Judges & Team Defense)

Use these exact analogies when explaining the project to non-technical judges or team members:

1. The “CEO and the Automated Assembly Line” (Why FPGA Acceleration?)

“Think of the microcontroller CPU as a brilliant CEO. If you force the CEO to manually inspect 50 raw optical heartbeat readings every second, they have to stop everything, do math, and burn battery. They become overwhelmed and have no time to make big decisions.
Instead, we used the FPGA fabric to build an **automated factory conveyor belt**. The raw light signals pass through our hardware filter and peak detector, which smooth the wave and count the pulse intervals in silicon. The hardware conveyor belt simply hands the CEO a finished note saying: ‘Heart rate is 128, IBI is 468 ms.’ Now the CEO’s entire brain is free to run our TinyML Neural Network.”

2. The “Formula 1 Wind Tunnel vs. The Pocket Commuter” (Why Two FPGAs?)

“The Xilinx Zynq-7000 was our Formula 1 wind tunnel: an industrial-grade testing platform where we proved our custom Verilog circuits were mathematically sound and closed timing at 69.45 MHz.
But you can’t give a \$250 development board to an agricultural worker in rural Bihar. The ShrikeFi platform (Renesas ForgeFPGA + ESP32-S3) is our production pocket commuter: it runs the exact same Verilog filter logic on a \$1.50 micro-FPGA, projecting the entire at-scale device cost under \$20 while retaining hardware acceleration.”

3. The “Smart Elevator Scale” (Why is the Filter $O(1)$?)

“Imagine calculating the average weight of 8 people inside an elevator. When a new person steps in and the oldest person leaves, a naive algorithm asks all 8 people to step on the scale again, adds up their weights, and divides by 8. That wastes time. Our $O(1)$ hardware filter acts like a smart scale: it remembers the previous running sum, subtracts the weight of the person who left, and adds the new person. It does this in a single 20-nanosecond clock cycle, regardless of how large the filter window is.”

4. The “4-Lane Walkie-Talkie” (How the ShrikeFi 4-bit Link Works)

“On large computer chips, components talk over 32 separate parallel copper tracks (like a 32-lane highway). On our compact micro-FPGA, we only have 4 data pins. We built a 4-lane high-speed walkie-talkie: when the ESP32 needs to send an 8-bit sensor reading, it sends the command code, slices the sample into two 4-bit ‘nibbles’, sends the first half, rings a digital doorbell (strobe), sends the second half, and rings the bell again. The FPGA snaps the two halves together in hardware in single-digit microseconds during software bring-up, and under 300 nanoseconds at maximum protocol clocking.”

5. The “Paramedic & The AI Chief Physician” (Dual-Core FreeRTOS Partitioning)

“On the ESP32-S3, Core 0 is our dedicated paramedic in the ambulance: it never leaves the patient, sampling optical vitals at 50 Hz and exchanging packets with the FPGA link without ever dropping a beat. Core 1 is the Chief Physician at the hospital: it takes the clean data, gathers environmental readings, runs the AI Neural Network, calculates disaster risk scores, and updates the wrist display.”

6. The “Teacher and the Student” (How the AI was Trained)

“Our clinical Rule Engine is a strict medical professor: 100% accurate according to published papers, but heavy and slow to calculate. Our Neural Network is an eager student. We gave the student 62,000 flashcards of extreme heatwaves and toxic smog scenarios. The professor graded every flashcard. By learning from its mistakes over 100 epochs of gradient descent, the student learned to make the exact same life-saving triage decisions in under 1 microsecond.”

9. LIKELY JUDGE QUESTIONS & WINNING DEFENSE ANSWERS

Q1: “Why did you build custom Verilog RTL instead of just doing everything in C on the ESP32-S3?”

- **Answer:** “Three reasons: Determinism, CPU offloading, and Power. Software signal processing on an RTOS suffers from interrupt latency and task jitter when WiFi or sensor interrupts fire. Our FPGA accelerator processes raw PPG samples at hardware clock speeds with zero CPU overhead, allowing the microcontroller to stay in low-power sleep modes between assessments.”

Q2: “How did you manage to fit your design into Renesas ForgeFPGA’s tiny 1120 LUT capacity?”

- **Answer:** “Our architecture was designed from day one to be ultra-lean. By using an $O(1)$ running-sum filter with bit-shift division and an accumulator-based peak detector, our entire ShrikeFi design consumes just **195 LUTs (17.41%)** and **110 flip-flops**, with zero DSP multiplier blocks and zero Block RAM. Over 82% of the FPGA remains free.”

Q3: “What makes your 4-bit link better than standard SPI or I2C?”

- **Answer:** “I2C is too slow (400 kHz) with heavy bus addressing, and SPI has protocol framing overhead. Our custom 4-bit parallel nibble link provides a dedicated hardware strobe and direct register-level latching. In our current bit-banged bring-up driver, a complete 32-bit IBI timestamp transfer takes only **18 microseconds** (occupying less than 0.1% of the 50 Hz optical sampling window). At our verified 10 MHz simulation target with hardware-assisted strobing, it transfers in **900 nanoseconds** (5.0 MB/s raw bandwidth) with zero protocol bloat.”

Q4: “What happens if the PM2.5 air quality sensor gets clogged or malfunctions?”

- **Answer:** “Our system uses cross-sensor validation. If the PMS5003 reports hazardous PM2.5 = 500 but the user’s blood oxygen is a healthy 99% and heart rate is 65 BPM, our sensor fusion engine recognizes the biometric mismatch, suppresses panic sirens, and flags a ‘Sensor Check Advisory’ on the OLED display.”

Q5: “How does this project transition to Qualcomm Snapdragon Wear?”

- **Answer:** “Our architecture is explicitly decoupled for platform migration. The Verilog DSP pipeline in `hardware/common/` maps directly onto Qualcomm’s Hexagon DSP core, while our C-based INT8 Neural Network executes natively on the Qualcomm Snapdragon Neural Processing Engine (SNPE) / NPU without architectural changes.”

10. CLINICAL REFERENCES & SCIENTIFIC FOUNDATIONS

Clinical Index	Scientific Source	Formulation & Thresholds in SIH26181
Steadman Heat Index	Robert G. Steadman (1979), “The Assessment of Sultriness”, Journal of Applied Meteorology.	Combines Ambient Temp + Humidity. Index > 35°C triggers Caution, > 40°C triggers Moderate; > 54°C triggers Critical Heat Hazard.
Cardio-Thermal Strain (CTSI)	Moran et al., Physiological Strain Index (PSI) adapted for multi-sensor edge wearables.	Fuses Heat Index with Cardiovascular Drift (HR > 130 BPM) and HRV collapse (RMSSD < 10 ms).
Pollution Respiratory Strain (PRSI)	WHO Global Air Quality Guidelines (2021) & EPA AQI Technical Assistance Document.	Fuses PM2.5 > 300 µg/m³ (EPA “Hazardous” ceiling) with clinical hypoxemia (SpO₂ < 88%).
Hypothermia & Cold Shock	Golden & Tipton, “Essentials of Sea Survival”, Cold Water Immersion Stages.	Tracks skin temperature proxy < 32°C (High) and < 28°C (Critical) combined with initial cold-shock tachycardia followed by severe bradycardia (HR < 50 BPM).

11. GLOSSARY OF TERMS

- **AXI4-Lite:** Advanced eXtensible Interface; standard memory-mapped point-to-point bus protocol for ARM SoC chips.
- **CTSI:** Cardio-Thermal Strain Index; our custom index quantifying physiological heat stress.
- **FSM:** Finite State Machine; a sequential digital hardware circuit transitioning between distinct operating states.
- **IBI:** Inter-Beat Interval; the exact elapsed time (in milliseconds or 20 ns clock ticks) between consecutive R-wave peaks.
- **INT8 Quantization:** Compressing 32-bit floating-point neural network weights into 8-bit signed integers for high-speed integer ALU execution.
- **Nibble:** A 4-bit aggregation of binary data (half an 8-bit byte).
- **PRSI:** Pollution Respiratory Strain Index; our custom index quantifying respiratory distress during smoke/smog events.
- **RMSSD:** Root Mean Square of Successive Differences; the clinical gold standard for measuring parasympathetic Heart Rate Variability (HRV).
- **SoC:** System on Chip; an integrated circuit combining CPU cores, memory, peripherals, and FPGA fabric.